



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE SIX: OBJECT-ORIENTED PROGRAMMING (OOP)

UNIT TWO: OBJECT-ORIENTED PROGRAMMING (OOP) PART II

OBJECT-ORIENTED PROGRAMMING (OOP) PART II

Learning Objectives

By the end of this unit, you will be able to:

- **explain and implement the concept of inheritance in object-oriented programming.**
- **apply polymorphism to write flexible and maintainable code.**
- **differentiate between method overriding and method overloading.**
- **use the super keyword to access and invoke parent class members effectively**

Learning Outcomes

At the end of this unit, you should be able to:

- **explain and implement inheritance.**
- **apply polymorphism to write flexibility code.**
- **differentiate between method overriding and overloading.**
- **understand and use the super keyword for accessing parent class members.**

Introduction

- **OOP is a powerful paradigm that models real-world entities using objects and classes. While the basics of OOP, such as classes, objects, and encapsulation, lay the foundation, this unit explores the advanced features such as inheritance and polymorphism.**
- **In this unit, you will learn how to create class hierarchies using inheritance to promote code reuse and organizational structure. You will also learn how polymorphism enables flexible and dynamic method execution, making your code more scalable and maintainable.**

What is Inheritance?

- **Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows a class (called the subclass, derived class, or child class) to inherit attributes (fields/variables) and behaviors (methods) from another class (called the superclass, base class, or parent class).**
- **This promotes code reuse, enhances scalability, and helps model real-world relationships in software design.**

Syntax of Inheritance

The *extend* keyword is used to indicate inheritance.

```
class Superclass {  
    // fields and methods  
}
```

```
class SubclassA extends Superclass {  
    // Additional fields and methods  
}
```

```
class SubclassB extends Superclass {  
    // Additional fields and methods  
}
```

Access Modifiers and Inheritance (1)

- Access modifiers control the visibility of class members (fields and methods) to other classes. A subclass can only access the fields and methods that are visible to it.
- The following are types of access modifiers:

Modifier	Class	Package	Subclass	Other
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default	Yes	Yes	No	No
private	Yes	No	No	No

Access Modifiers and Inheritance (2)

- “**public**” members are accessible everywhere.
- “**protected**” members are accessible within the same package and subclasses even if they are in different packages.
- No modifier (default) limits access to within the same package.
- “**private**” members are not accessible in child classes.

Usage of public Access

```
class Animal {  
    public void eat() {  
        System.out.println("Animal eats food");  
    }  
}  
  
class Dog extends Animal {  
    public void bark() {  
        eat(); // Accessible because eat() is public  
        System.out.println("Dog barks");  
    }  
}  
  
public class AnimalTest {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

- Since ***eat()*** is ***public***, it can be accessed in any subclass or even other classes outside the ***package***.

- The code will output:

Animal eats food

Animal eats food

Dog barks

Usage of protected Access

```
class Animal {  
    protected void sleep() {  
        System.out.println("Animal sleeps");  
    }  
}  
  
class Cat extends Animal {  
    public void rest() {  
        sleep();  
        System.out.println("Cat rests");  
    }  
}  
  
public class AnimalTest {  
    public static void main(String[] args) {  
        Cat myCat = new Cat();  
        myCat.rest();  
    }  
}
```

- In the code, the ***protected*** access modifier allows the ***Cat*** subclass to access the ***sleep()*** method of its parent class ***Animal***.
The code will output:

Animal sleeps

Cat rests

Usage of Default Access Modifier

```
class Animal {  
    void move() {  
        System.out.println("Animal moves");  
    }  
}  
  
class Tiger extends Animal {  
    public void run() {  
        move(); // move() is accessible as Tiger  
                // is in the same package with Animal  
        System.out.println("Tiger runs fast");  
    }  
}  
  
public class AnimalTest {  
    public static void main(String[] args) {  
        Tiger tiger = new Tiger();  
        tiger.run();  
    }  
}
```

- Since both *Animal* and *Tiger* classes are in the same *package*, the code will output.

Animal moves

Tiger runs fast

- If they are in different packages, it will cause a compile-time error.

Usage of private Access

```
class Animal {  
    private void breathe() {  
        System.out.println("Animal breathes");  
    }  
    public void showBreathing() {  
        breathe(); // Accessible within the same class  
    }  
}  
  
class Horse extends Animal {  
    public void display() {  
        // breathe(); // Error: Cannot access  
        showBreathing(); // Allowed through public method  
    }  
}  
  
public class AnimalTest {  
    public static void main(String[] args) {  
        Horse myHorse = new Horse();  
        myHorse.display();  
    }  
}
```

- “**private**” members can be accessed indirectly through **public** or **protected** methods.
- In the code, the **private** method **breathes()** is accessed indirectly via the public method **showBreathing()**.

Types of Inheritance

The following are the types of inheritance:

- **Single Inheritance**
- **Multilevel Inheritance**
- **Hierarchical Inheritance**

Single Inheritance

- A subclass inherits from one superclass. In the code below, the *Dog* class inherits *eat()* from the *Animal* class. This means any Dog object can call both the *eat()* and *bark()* methods.

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eat...");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is bark...");  
    }  
}
```

Multilevel Inheritance

- In multilevel inheritance, a class is derived from a derived class.
- In the code, *Puppy* class inherits features from the *Dog* class and by extension from the *Animal* class (the *eat()* method) without duplicating code.

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}  
  
class Dog extends Animal { // Level 1  
    void bark() {  
        System.out.println("Barking...");  
    }  
}  
  
class Puppy extends Dog { // Level 2  
    void weep() {  
        System.out.println("Weeping...");  
    }  
}
```

Hierarchical Inheritance

- Multiple subclasses inherit from a single class. In the code, the both *Lion* and *Cat* inherit *eat()* method from the *Animal* class.

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}  
  
class Cat extends Animal {  
    void meow() {  
        System.out.println("Meowing...");  
    }  
}  
  
class Lion extends Animal {  
    void roar() {  
        System.out.println("Roaring...");  
    }  
}
```

Using *super* Keyword

- The *super* keyword is a special reference variable used within a subclass to refer directly to its immediate superclass. It plays a crucial role in inheritance, allowing a subclass to access members (variables and methods) and constructors of the superclass. There are two primary uses of *super*:
 - Calling the Superclass Constructor
 - Accessing Superclass Methods

Calling the Superclass Constructor (1)

- If the super class constructor is not called explicitly, Java *automatically invokes* the superclass with no-argument constructor.

- The code will output:

Parent constructor

Child constructor

```
class Parent {
    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {
    Child() {
        // Java automatically calls super() here
        System.out.println("Child constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
    }
}
```

Calling the Superclass Constructor (2)

If the superclass does not have a ***no-arg constructor***, the subclass must explicitly call a parameterized constructor using `super(arguments)` as shown in the code.

```
class Parent {
    Parent(String name) {
        System.out.println(
            "Parent constructor. Name: " + name);
    }
}

class Child extends Parent {
    Child(String name) {
        super(name); // Must call param. constructor
        System.out.println("Child constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        new Child("Alice");
    }
}
```

Accessing Superclass Methods (1)

- When a subclass overrides a method from its superclass, it provides its own version of that method. Sometimes, inside the subclass, you might still want to call the original (superclass) version of the method. The keyword *super* lets you do this

super.methodName();

- This calls the superclass version of the method, even if it is overridden in the subclass.

Accessing Superclass Methods (2)

- In the code, ***start()*** is overridden in the ***Car*** class. Using ***super.start()*** allows the subclass to call the superclass version of the method.
- The output is:

Car is starting
Vehicle is starting

```
class Vehicle {  
    void start() {  
        System.out.println("Vehicle is starting");  
    }  
}  
  
class Car extends Vehicle {  
    void start() {  
        System.out.println("Car is starting");  
    }  
  
    void display() {  
        start();        // Calls Car's start()  
        super.start();  // Calls Vehicle's start()  
    }  
}  
  
public class TestSuperMethod {  
    public static void main(String[] args) {  
        Car c = new Car();  
        c.display();  
    }  
}
```

What is Polymorphism?

- Polymorphism is the ability of different objects to respond to the same method in different ways. It allows objects to be treated as instances of their parent class rather than their actual class.
- For instance, suppose we have the hierarchy, *Animal* (parent class) and a *Dog* class (child class) that extends *Animal*, we can write:

Animal myAnimal = new Dog()

- Here, the actual object is a *Dog*, but the reference variable is of type *Animal*.

Types of Polymorphism

There are two main types of polymorphism in Java:

- i. Compile-time (static) polymorphism**
- ii. Runtime (dynamic) polymorphism**

Compile-time Polymorphism (1)

- Compile-time polymorphism is achieved through ***method overloading***. Method overloading occurs when multiple methods in the same class share the same ***name*** but have ***different parameters*** (***different types***, ***number*** or ***both***).
- The method to be called is resolved during compilation
- Methods have the same name but different parameter lists.

Compile-time Polymorphism (2)

- In the code, the ***add()*** method is overloaded three times with different parameter lists.
- The compiler determines which method to call based on the arguments.

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.add(5, 10));  
        System.out.println(calc.add(5.5, 6.3));  
        System.out.println(calc.add(1, 2, 3));  
    }  
}
```

Runtime Polymorphism (1)

- Runtime polymorphism is achieved through **method overriding**. Method overriding happens when a subclass provides its own implementation of a method declared in its superclass.
- The method to be called is resolved at runtime.
- The method signature (**name, return type, parameters**) must be the same.

Runtime Polymorphism (2)

- In the code snippet, ***Appliance*** is the superclass while ***WashingMachine*** and ***Microwave*** are subclasses that override the ***operate()***.

```
class Appliance {  
    void operate() {  
        System.out.println("Appliance is operating");  
    }  
}  
  
class WashingMachine extends Appliance {  
    @Override  
    void operate() {  
        System.out.println("Washing Machine is washin");  
    }  
}  
  
class Microwave extends Appliance {  
    @Override  
    void operate() {  
        System.out.println("Microwave is working");  
    }  
}
```

Runtime Polymorphism (3)

- Although *myAppliance* is of type *Appliance*, it behaves differently at runtime depending on the object assigned to it.

```
public class Main {  
    public static void main(String[] args) {  
        Appliance myAppliance;  
  
        myAppliance = new WashingMachine();  
        myAppliance.operate();  
  
        myAppliance = new Microwave();  
        myAppliance.operate();  
  
        myAppliance = new Appliance();  
        myAppliance.operate();  
    }  
}
```

Polymorphism with Abstract Class (1)

- **Polymorphism is commonly used with abstract classes and interfaces, where methods are declared but not implemented in the parent type and implemented in subclasses.**
- **When using abstract classes, polymorphism works by having multiple classes inherit from a common abstract base class, then override its methods.**

Polymorphism with Abstract Class (2)

- In the code, *Animal* is an abstract class with an abstract method *makeSound()*. *Dog* and *Cat* extend *Animal* and provide their own implementation.
- At runtime, the appropriate method is called based on the object type.

```
abstract class Animal {  
    abstract void makeSound();  
}  
  
class Dog extends Animal {  
    public void makeSound() {  
        System.out.println("Dog: Woof!");  
    }  
}  
  
class Cat extends Animal {  
    public void makeSound() {  
        System.out.println("Cat: Meow!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal myAnimal1 = new Dog(); // Polymorphism  
        Animal myAnimal2 = new Cat(); // Polymorphism  
  
        myAnimal1.makeSound(); // Output: Dog: Woof!  
        myAnimal2.makeSound(); // Output: Cat: Meow!  
    }  
}
```

Polymorphism with Interfaces (1)

- **Interfaces provide a way to achieve polymorphism by defining a contract (methods) that multiple classes can implement differently.**
- **When you use an interface reference to point to different implementing objects, you can invoke methods polymorphically.**

Polymorphism with Interfaces (2)

- The code uses a *Vehicle* interface with a *move()* method. *Car* and *Boat* implement it with their own behavior.
- The *main()* method shows polymorphism by calling *move()* on different objects through a common interface.

```
abstract class Vehicle {  
    abstract void move();  
}  
  
class Car extends Vehicle {  
    public void move() {  
        System.out.println("Car moves on roads.");  
    }  
}  
  
class Boat extends Vehicle {  
    public void move() {  
        System.out.println("Boat sails in water.");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Vehicle v = new Car();  
        v.move();  
  
        v = new Boat();  
        v.move();  
    }  
}
```

In-Text Questions (ITQs)

- 1. What is inheritance in Object-Oriented Programming?**
- 2. What are the main benefits of using inheritance?**
- 3. When should you use an interface instead of an abstract class?**

In-Text Answers (ITAs)

- 1. Inheritance is a feature in OOP where a class (called the subclass) inherits properties and methods from another class (called the superclass). This allows the subclass to reuse code from the superclass, promoting modularity and easier maintenance.**
- 2. The main benefits include code reusability, extensibility, easier maintenance, and the ability to model real-world hierarchical relationships logically.**
- 3. Use an interface when you want to define a contract that can be implemented by unrelated classes. Use an abstract class when classes share some common functionality and structure.**

Conclusion

- **Understanding inheritance and polymorphism is crucial for mastering Object-Oriented Programming. These concepts enhance code reuse, simplify maintenance, and allow for dynamic method behavior, making programs more flexible and easier to extend.**
- **The use of the super keyword helps maintain clear relationships between classes, while polymorphism enables writing generalized yet adaptable code. Mastery of these advanced OOP features prepares you to design robust and scalable applications that effectively model real-world scenarios.**

Summary

- **This unit focused on inheritance and polymorphism. Inheritance lets a subclass reuse and extend a superclass's properties and methods, supporting code reuse, extensibility, and hierarchical design. You explored single, multilevel, and hierarchical inheritance, and used the super keyword to access superclass members.**
- **Polymorphism allows different objects to respond to the same method call in unique ways. The unit explained compile-time polymorphism through method overloading and runtime polymorphism via method overriding. It also highlighted how abstract classes and interfaces enable more flexible and scalable code.**